

Course Project

Phase #2. IR Generation

Prof. Jaeseung Choi

Dept. of Computer Science and Engineering

Sogang University

General Information

- **Check "Project #2" in *Assignment* tab of *Cyber Campus***
 - Skeleton code (`Prj2.tgz`) is attached in the post
 - Deadline: **12/3** Wednesday 23:59
 - Submission will be accepted in that post, too
 - Late submission due: **12/5** Friday 23:59 **(-20% penalty)**
- **Please read the instructions in this slide carefully**
 - The specification of the project is quite complex
 - The slide also contains important submission guidelines
 - If you do not follow the guidelines, **you will get penalty**

(Remind) LLM Policy

- **You may get the help from LLMs like ChatGPT**
 - But first try to write the code by yourself
 - Ask LLMs only after that, when you really don't have a clue
 - If you don't practice F# programming at this stage, you will have a big trouble in the last phase
- **Again, you must submit a report if you used LLMs**
 - Write the exact **prompt you entered** to an LLM
 - Also, include the **code you obtained** from that prompt and **how/why you modified** that code
 - If you submit LLM-generated code **without a report**, it will be **considered as cheating**

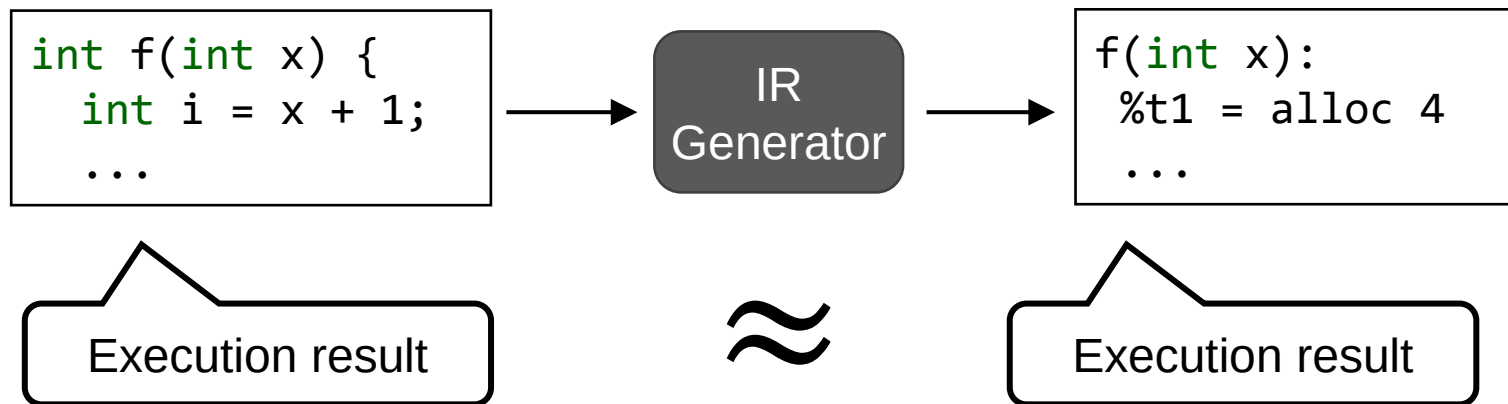
Outline

■ In this phase, you will implement AST-to-IR translator

- Let's only care about the correctness of translation in this phase (we will think about efficiency and optimization later)
- Generated IR must behave equivalently to the source program
- In other words, two functions below must return the same value

■ How can we test the correctness in this project?

- IR executor (interpreter) is provided in the skeleton code



Skeleton Code (Same as before)

- Copy Prj2.tgz into CSPRO server and decompress it
 - You must connect to csproun.sogang.ac.kr (N = 2 or 3)
 - Don't decompress-and-copy; copy-and-decompress
- **IRGenerate.fsproj**: F# project file (you may ignore)
- **src/**: Source files you have to work with
- **testcase/**: Sample test cases and their answers
- **check.py**: Script for self-grading with test cases
- **config**: Used by the grading script (you may ignore)

```
jschoi@csprou2:~$ tar -xzf Prj2.tgz
jschoi@csprou2:~$ ls Prj2/
IRGenerate.fsproj  check.py  config  src  testcase
```

Structure of src Directory

- **AST.fs** : Definition of Mini-C program AST
- **Lexer.fsl & Parser.fsy**: Input files for F# Flex/Bison
- **Main.fs** : The main driver code (explained later)
- **IR.fs** : Definition of IR code (= list of instructions)
- **Translate.fs** : You must write the translator code here
 - **Helper.fs** contains some helper functions that you can use
- **Executor.fs & State.fs**: IR executor (interpreter)

```
jschoi@cspro2:~/Prj2$ cd src/  
jschoi@cspro2:~/Prj2/src$ ls  
AST.fs          Helper.fs  Lexer.fsl  Parser.fsy  Translate.fs  
Executor.fs     IR.fs     Main.fs    State.fs
```

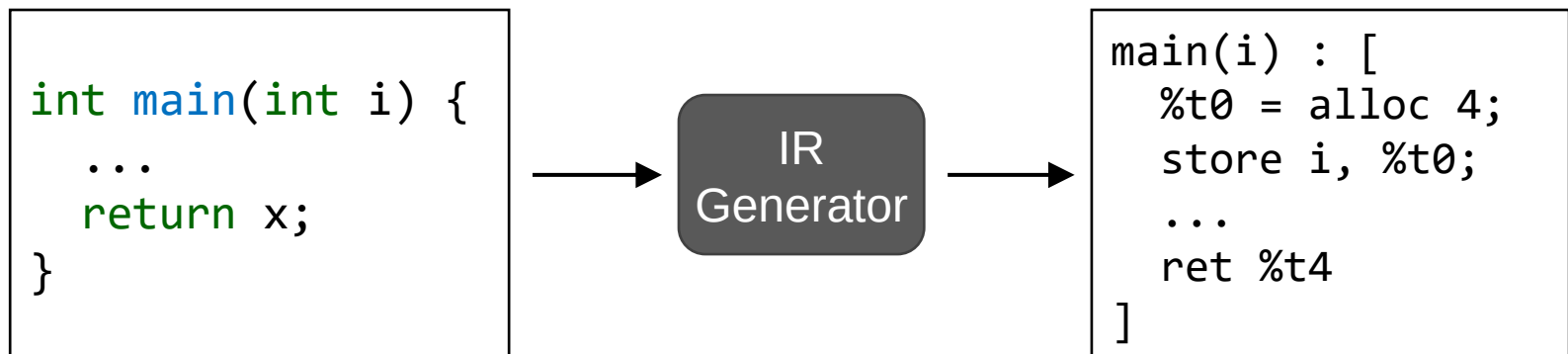
Structure of IR Code

■ Our IR code will also consist of one function

- IR function consists of name, argument, and **instruction list**
- Generating this instruction list is the key challenge of the project
- What should we do for the arguments? I will explain later

■ Read IR.fs file to see what kind of instructions are available in our IR language

- Read the comments in the file carefully, too



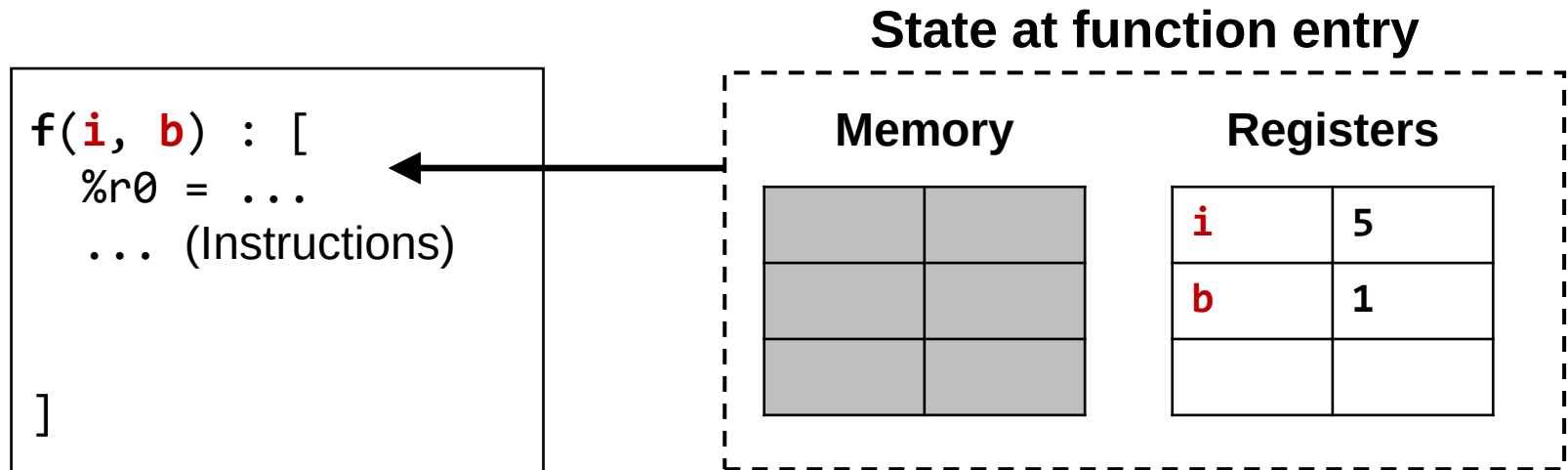
IR Execution Model

- As we assumed in our lecture slide, the state of IR execution will consist of a register map and a memory
 - Register map is a mapping from register name to value
 - Register has 4-byte width (i.e., word size = 4 byte)
 - Memory is a mapping from address to value and our IR supports 1-byte access and 4-byte access
- Also, recall that all values are *integers* in our IR model
 - **true** must be translated to 1 and **false** must be translated to 0

	Memory		Registers	
(For x)	1000	2	t1	2
(For y)	1004	5	t2	5
	...	...	...	...

Passing Arguments

- **Note that a function in IR code also takes in arguments**
 - Are these arguments passed through registers or memory?
- **It's a matter of choice: we will follow the LLVM model**
 - Assume that arguments are passed through register
 - At the function entry, argument registers are initialized with input values (register **i** and **b** in this example)



Changes to Mini-C Language (★)

- In this phase, assume that function arguments and return values will always have **bool** or **int** types
 - For example, function like `f(int *p)` is a valid program but will **not** be used as a test case (it's difficult to test such functions)
- From now on, our Mini-C language will have *array*
 - But limited to 1-dimensional array of **bool** and **int**
 - Ex) `bool b[32]; int i[32];`
- Array names can be assigned to a pointer variable
 - Also, we will allow *add operation* on a pointer
 - In addition, asterisk (*) operator can now appear in front of any expression (check the changes made to the `AST.fs` file)
 - Ex) `int arr[32]; int *p = arr; *(p + 1) = 2; // Valid`

Specification for IR Translation

- During the IR translation, you can assume that the input program is **error-free**
 - No type mismatch, NULL dereference, division-by-zero, etc.
 - Discussion: Is it okay to make such assumption?
- We will assume that our source language defines the semantics of **&&** and **||** with short-circuit evaluation
 - Therefore, your translator must generate IR code that implements such short-circuit evaluation
 - For example, the IR translated from the following **Mini-C** program must not raise a div-by-zero error at runtime

```
if (y != 0 && x / y >= 2) {  
    return x / y;  
}
```

You Mission: Translate.run

- In the **Translate.fs** file, you have to implement the following function (its type must not change)
 - **let run (prog: Program) : IRCode = ...**
 - This function must translate **Mini-C** program into **IRCode** (see **IR.fs** to check the definition of **IRCode**)
- I already provided some code as a guideline
 - Key functions are declared with appropriate type and comment
 - You will have to add many more functions to complete
 - FYI, my reference solution is about 300 lines

Three Modes in Main Driver

- In this phase, there are three modes you can run
 - **print-ast** mode: Same as **Phase #1**
 - **print-ir** mode: Generate IR with your translator and print that IR code (you can use this for debugging)
 - **run-ir** mode: Generate IR with your translator and run that IR code on the provided executor

```
jschoi@cspro2:~/Prj2$ dotnet build -o out
...
jschoi@cspro2:~/Prj2$ ./out/IRGenerate
<Usage>
[*] ./out/IRGenerate print-ast <source file>
[*] ./out/IRGenerate print-ir <source file>
[*] ./out/IRGenerate run-ir <source file> <input file>
```

Running the Translated IR

- Using the **run-ir** mode, you can run the generated IR code and check its execution result (return value)
 - Note that you have to provide the **input file** as an argument

```
jschoi@cspro2:~/Prj2$ cat testcase/inp-1
```

```
30 1  
30 0  
...
```

Each line means **f(30,true)** and
f(30,false) respectively

```
jschoi@cspro2:~/Prj2$ ./out/IRGenerate run-ir testcase/prog-1 \  
testcase/inp-1
```

```
61  
14  
...
```

Input file

Self-Grading

- Once you think everything is done, you can run the **check.py** script as a final check
 - 'O': Correct, 'X': Incorrect, 'E': exception, 'C': Compile error, 'T': Timeout (maybe infinite recursion)
- Recall that I use different test cases in the real grading
 - So it's important to think of your own test inputs and check your code with them
- You can assume the test cases will be free from error

```
jschoi@cspro2:~/Prj2$ ./check.py  
[*] Result : 00XXX
```

Tip #1

- What should be the return type of `trans_exp(e)`?
- In the lecture note, we promised that the last instruction of `trans_exp(e)` defines a register with the value of e
- In the skeleton code, I defined `trans_exp(e)` in a slightly different way (to make programming easier)
 - It returns a pair of **register name** and **instruction list**
 - Ex) `let (r, instrs) = transExp symtab e in ...`
 - Here, **r** is the register that contains the result of computing e : This register doesn't have to be defined by the last instruction anymore, it can be defined by an instruction at the middle

Tip #2

■ Handling of pointer and array can be tricky

- For this, `trans_exp(e)` and `trans_stmt(e)` will have to consider several corner cases
- Note that these functions may have to be defined slightly different from our lecture note

■ Start from easy cases and extend your implementation step by step

■ Also, don't forget to create your own test cases to test your implementation

Tip #3

- You must write a function that **generates IR code**, not a function that **executes IR code**
 - These two things are completely different, but many students get confused in this point
 - For example, when writing `Translate.run()`, you don't have to call functions like `Executor.run()` or `Executor.execInstr`
 - Remember that your goal is to generate IR code correctly; you run this IR on the executor only to test whether it is correct

Tip #4

- **What should I do when the IR code fails to run or produce wrong outputs?**
- **There are many possibilities (consider each one by one)**
 - First, there may be a mistake in your translation logic
 - Print the generated IR and check if it looks fine
 - Second, you may be misunderstanding instruction's meaning
 - Read the definition of instructions in **IR.fs**
 - Lastly, there may be a bug in the provided executor code
 - In other words, I might have made a mistake ☹️
 - Please let me know if the executor seems to have a bug

Submission Guideline

- You should submit one F# source code file and report
 - `Translate.fs`
 - `report.pdf` (If you used LLM)
- The whole project **must properly compile** when I copy the `Translate.fs` file you submitted
 - If the skeleton code does not build, **cannot give you any point**
- **Submission format**
 - Upload these files directly to *Cyber Campus* (**do not zip them**)
 - **Do not change the file name** (e.g., adding any prefix or suffix)
 - If your submission format is wrong, you will get **-20% penalty**